11 — Guides & FAQs

11 — Guides & FAQs

Revision: 1 **Last modified:** 2026-07-05T14:20:00Z **Status:** Draft — consolidated from the four phase WBS docs, `v07-execution/*`, and `v06-deploy/disaster-recovery.md`.

---

1. Position

This section is the **practical companion** to the rest of the consolidated implementation source of truth. It gives a high-level roadmap of the four-phase programme, onboarding checklists for engineers and operators, operational runbooks, and answers to frequently asked questions.

For authoritative detail, follow the cross-references back to the phase WBS docs and the Volume-6/7 nano-detail docs.

---

2. Four-phase roadmap at a glance

Phase	Goal	Exit gate / definition of done
0 — Spike	Settle make-or-break risks on production interfaces	G1–G6 gates pass; surviving interfaces feed Phase 1
1 — MVP	Self-hostable overlay network with privacy-VPN front end	8 acceptance criteria + 4 SLOs green
2 — Parity + Reach	Mullvad-parity transport set, P2P/NAT, PQ, DAITA, HA	P2-AC1..8 + P2-SLO1..5 green
3 — Extended Reach	HarmonyOS, Aurora, WASM proxy, billing, audit, reproducible builds	G20–G26 gates + release tag

Critical path summary (risk-ordered, sizing-only TARGETs):

- **Phase 0:** longest chain `Transport trait → plain-UDP → orchestrator → G1 → MASQUE → G2 → edge-language benchmark G4`.
- **Phase 1:** `monorepo → FORCE-RLS → PKI device-cert → coordinator+WatchNetworkMap → transport/kill-switch → edge → QA/DoD cert → MVP tag`.
- **Phase 2:** `HA/multi-region → STUN discovery → hole-punch → relay → multi-hop → P2 QA/DoD → parity tag` (PQ and DAITA run in parallel).
- **Phase 3:** `fork runners → native shims → third-party audit → l10n/release → reach tag`, with E26 reproducible builds running in parallel.

For the full dependency DAG, see ../../v07-execution/dependency-graph.md.

---

3. Onboarding checklists

3.1 New engineer — first day

1. Read 00 — Executive Summary and 02 — System Architecture.
2. Confirm the toolchain:
   - Go 1.25+, Rust 1.85+, Node/pnpm for `helix_design`, Flutter 3.29+.
   - `buf`, `sqlc`, `podman` (rootless), `make`.
3. Run `make gen` and `make verify-gen` to confirm schema-first codegen is clean.
4. Read the phase WBS doc for the area you will work in (`06-phase0-spike-wbs.md` … `09-phase3-reach-wbs.md`).
5. Open one nano-detail doc in your area (e.g. `v02-data-plane/transport-trait.md` or `v03-control-plane/svc-coordinator.md`).
6. Check `docs/workable_items.db` and find your first item via `cmd/workable-items` (work-item `HVPN-P1-150`).

3.2 New operator — first gateway

1. Install rootless Podman and confirm `net.ipv4.ip_unprivileged_port_start=443`.
2. Run `helixvpnctl init --domain gw.example.com`.
3. Review the generated quadlet units under `~/.config/containers/systemd/`.
4. Start the stack: `systemctl --user start helixvpn-pod`.
5. Create an enrollment token: `helixvpnctl enroll-token --tenant example --ttl 1h`.
6. Enroll a Connector and a Client, then verify reach to an authorized LAN host.
7. Read the DR runbook in ../../v06-deploy/disaster-recovery.md.

---

4. Operational runbooks

4.1 Disaster recovery

HelixVPN has exactly one durable source of truth: Postgres. Everything else is ephemeral or regenerable.

Asset	Backed up?	Why
Postgres + WAL	Yes	Only durable truth
CA root / PKI secrets	Yes, KMS-encrypted, offline	Catastrophic if lost or leaked
Redis / NATS	No	Ephemeral; backing them up would create a connection log (C3 violation)
Deploy manifests	No	Regenerated via `helixvpnctl deploy`

Targets (§11.4.6 — not guarantees until drills measure them):

- Postgres restore RTO < 15 min; RPO ≈ WAL archive interval.
- Whole-region failover RTO < 15 min; RPO = 0 with synchronous standby.

Steps for a single-store recovery:

1. Snapshot the current (even corrupt) store before overwriting.
2. Record the target recovery point and expected post-restore state.
3. Restore to a **fresh** volume/cluster; never overwrite the live primary in place.
4. Run the verification gate: schema version, RLS intact, `schemalint` green, row-count sanity, live `WatchNetworkMap` snapshot.
5. Promote only after all gate checks pass.

See the full runbook at `../../v06-deploy/disaster-recovery.md`.

4.2 Dependency graph

Every cross-phase dependency is transcribed from the `depends_on`/`deps` fields in the four WBS docs; nothing is invented. The `workable-items` loader enforces DAG acyclicity.

Key rules:

- **Risk-ordered sequencing:** correctness floor (RLS, no-log lint, kill-switch, NAT honesty, PQ downgrade-safety) leads; convenience UI trails.
- **Disjoint-scope PWUs run in parallel.**
- **Single-resource-owner:** shared hardware (e.g. bench host) is never double-booked.
- **Device-gated Phase-3 work parks** on `PENDING_DEVICE` while host-only work continues as ≥3 background streams.

4.3 Workable-items model

The WBS is bidirectionally synced to the git-tracked SQLite database `docs/workable_items.db` via Docs Chain.

- Every leaf carries `HVPN-Pn-NNN[.k]` id, status, type, description, acceptance, effort, tests, deps.
- A `PASS` in the per-item test diary is impossible without a captured evidence path (schema `CHECK`).
- `validate` blocks commit if md↔db drift or a dependency cycle exists.

See `../../v07-execution/workable-items-model.md`.

---

5. FAQs

FAQ-1 — Where is the single source of truth?

The consolidated implementation docs under `docs/research/mvp/final/implementation/` are the navigable source of truth. If a detail is missing here, read the owning nano-detail doc in the parent `final/` tree.

FAQ-2 — Which language owns which layer?

Layer	Language	Reason
Control plane	Go	Mandated operator stack; mature ecosystem (Gin, Connect-RPC, sqlc)
Data plane / edge	Rust	Byte-for-byte reuse with client core; iOS memory ceiling
Client core	Rust + Flutter	Shared core across 8 platforms; Flutter reaches iOS/Android/ HarmonyOS/Aurora
UI tokens		

Layer	Language	Reason
	JSON → polyglot	One source → CSS/Dart/Swift/ Compose/ArkTS/C-Qt

FAQ-3 — Why not sing-box or Hysteria2 as the primary transport?

Hysteria2/Salamander and Shadowsocks are reflected in the Phase-2 transport set, but the primary obfuscation is **MASQUE/ QUIC = WireGuard-over-HTTP/3** (RFC 9298), matching Mullvad's production mechanism. The custom Rust `helix-transport` crate shares code between client and edge and fits the iOS memory budget better than a Go framework would.

FAQ-4 — What happens if the iOS Network Extension memory gate fails?

G3 is make-or-break. If the Rust core cannot fit in the iOS NE budget, the iOS shim is re-planned against the fallback ladder in `07-phase1-mvp-wbs.md` §5 — surfaced as an operator decision, never silently hidden.

FAQ-5 — How is "no logging" enforced?

- Schema lint (`schemalint`) forbids connection/traffic tables.
- Redis presence keys have TTLs and are never persisted.
- Telemetry is counts/gauges only; no `tenant_id`, `device_id`, `src_ip`, `dst_ip` labels.
- DR explicitly does **not** back up Redis/NATS because restoring them would create a connection log.

FAQ-6 — How do I add a new transport?

1. Implement the `Transport` trait in `helix-transport`.
2. Add a carrier variant and MTU accounting.
3. Wire it into the client/edge orchestrator and the transport ladder.
4. Add test types: `UNIT`, `INT`, `E2E`, `STRESS`, `SEC` (and `CHAL` if a Challenge exists).
5. Update `08-phase2-parity-wbs.md` if it is a parity transport, or `07-phase1-mvp-wbs.md` if it is MVP-critical.

FAQ-7 — Why are there no date commitments in the WBS?

Every duration is a sizing-only `TARGET`. Constitution §11.4.6 and the phase WBS docs state explicitly that effort figures are "indicative person-day totals … **not** a commitment". Calendar dates require operator planning and are not authored in the spec.

FAQ-8 — What is the RBAC role matrix?

Roles are defined in the product scope and enforced by the identity/API services. FR-610 gives RBAC a dedicated requirement: a `member`/`operator` principal cannot invoke actions restricted to a higher role. See 01 — Product Scope and `v03-control-plane/svc-identity.md`.

FAQ-9 — How do local connector ACLs interact with central policy?

Consolidation decision (pending coordinator confirmation):

1. **Local-deny overrides central-allow** — a connector can tighten its own network's policy.
2. **Central-deny overrides local-allow** — tenant-wide default-deny/fail-closed wins.
3. The connector advertises its local-deny list to the coordinator so the edge enforces it consistently.

See 03 — Data Plane §9 and 04 — Control Plane §6.

FAQ-10 — Where do I report a documentation inconsistency?

Open a finding in `docs/reviews/mvp-final/findings/` and, if it is a real contradiction, add a row to `99-source-coverage-ledger.md` gaps. Do not silently resolve cross-doc conflicts in the consolidated docs alone.

---

6. Cross-references

- Phase 0 WBS → `../../06-phase0-spike-wbs.md`
- Phase 1 WBS → `../../07-phase1-mvp-wbs.md`
- Phase 2 WBS → `../../08-phase2-parity-wbs.md`
- Phase 3 WBS → `../../09-phase3-reach-wbs.md`
- Dependency graph → `../../v07-execution/dependency-graph.md`
- Workable-items model → `../../v07-execution/workable-items-model.md`

- Disaster recovery → `../../v06-deploy/disaster-recovery.md`
- Testing & QA → 09 — Testing & QA

Sources: `06/07/08/09-phase*-wbs.md`, `v07-execution/*.md`, `v06-deploy/disaster-recovery.md`.